

Optimized Maximum Intensity Projection (MIP)

Georgios Sakas, Marcus Grimm, Alexandros Savopoulos

Fraunhofer Institute for Computer Graphics
Wilhelminenstr. 7, 64283 Darmstadt, Germany
email: {gsakas,mgrimm}@igd.fhg.de

Abstract We present an improved version of the MIP algorithm that requires roughly 2%-10% of the computational effort of the brute-force, straight-forward version. The algorithm conserves the image quality and requires no pre-processing. We propose four different quality levels: a fast line-traversing nearest neighbour algorithm for previewing, two efficient approximation algorithms that are sufficient for most applications, and an analytical method for images of highest quality. Additionally we present an improved cache memory access scheme. The employed 'sub-cube' volume data representation provides an additional speed up of at least 50% compared to the classical linear memory access. This is of particular interest for (but not restricted to) parallel volume rendering on shared memory multiprocessor systems.

1 Motivation

Maximum Intensity Projection (MIP) is widely accepted as a volumetric visualization method capable of improving the perception of the location, shape, and topology of vessels, and therefore is supported by all major industrial vendors of MRA devices [Vand91]. The main idea is surprisingly simple: due to the acquisition conditions, vessels result in a 'brighter' spot than their surroundings on a slice. Thus, the maximum value detected along a straight line through the volume will most probably belong to a vessel rather than to other tissue. If the maxima of all rays going through all pixels are projected on the screen, the all-around shape of vessels becomes visible. Although 'stronger' vessels may superpose thin ones and it is not possible to say which vessel is in front and which is in the back, the impression for the global vessel shape is improved significantly.

The disadvantage of the method is that the search for the maximum along a ray can become a very time consuming process. A typical MRA dataset has a resolution of $256^2 \cdot 128$, that is 8 MVoxels. If an image has the size of 256^2 pixels, assuming that on average about 80% of the rays will hit the data, approximately 50,000 rays need to be calculated. If each of these rays traverses on average 200 voxels, a total number of $50,000 \cdot 200 = 10$ million locations within the volume have to be processed, resulting in 10 million voxel accesses and evaluations even in the simplest case (nearest neighbour, no interpolation, one access per voxel, no super sampling). If tri-linear interpolation is employed, the above mentioned complexity increases by a factor of at least 4. Due to the lack of depth information in a MIP image, a small animation loop is usually appreciated, which means that 5-20 such images have to be computed. Obviously, without significant optimization such a straight-forward, brute-force method can only be implemented on large expensive

computers that are not available to most users.

2 Problem Analysis

The most striking fact in the above mentioned calculation is that most of the computational effort does not conclude to the final result: Since only one maximum value is projected per ray, all other computed values are overwritten. Thus, less than 1% of the calculated values are indeed represented on the screen, a fact indicating significant potential for optimizations.

As analyzed in [Saka93d], the volume rendering process can be divided into four stages: traversing, sampling, illumination, and accumulation. Due to the nature of MIP, illumination is trivial and accumulation does not take place. Thus, optimizations have to be performed on the remaining stages that are traversing and sampling.

The task in the 'traversing' stage is to find all voxels contributing to a ray effectively. In the case of nearest neighbour access, those are all voxels hit by the 3D line that are found by using a fast 6-connected DDA algorithm ([ClWy88], [Slat92], [YaCK92]). However, due to the trilinear interpolation, voxels not directly hit by the line may also contribute to the pixel as well, see fig. 1. Finding these additional voxels requires extra work. The first optimization is to develop a fast line traversing method addressing only the voxels that contribute to the ray density.

With regard to 'sampling', an important observation mentioned above is that except for one, all other calculated values along the ray are overwritten. Since sampling employs trilinear interpolation – a very computational demanding operation –, the data could be pre-selected using an inexpensive, fast filter and sampled only at locations with promising success. In order to maintain interactivity, pre-processing of the data must be either avoided completely, or become very fast.

The implication of an improved sampling speed is that the access to the voxels stored in the main memory must be accelerated as well. Maximizing the cache coherency is becoming increasingly important when the average computations per voxel decrease.

A final observation deals with the method used to identify the exact position and value of a maximum candidate. Due to the employed trilinear interpolation, density values are not constant, but vary within the 3D space between voxel midpoints. Thus, the exact position and value of the maximum within 3D space is unknown and has to be computed. The straight-forward idea to oversample the complete ray requires prohibitive much computation time. A possible improvement is to first identify locations of interest and then to oversample these locations using either a uniform sampling or a binary search adaptively. In both cases oversampling remains computational demanding and the calculated value still is an approximation. Instead, a method is needed that provides good results while avoiding 'unsmart' oversampling with the corresponding interpolations.

In summary, four possible optimizations have been identified:

1. Addressing all voxels contributing to a ray, i.e. affected by trilinear interpolation. Effective ray-traversing by exploiting coherency, integer arithmetic etc.
2. Avoid unnecessary interpolations along the ray, evaluate only at locations promising success.
3. Find the accurate position and value of the maximum without oversampling for these selected locations.

4. Minimize the voxel access time by maximizing the cache coherency.

3 Line Traversing in Voxel Space

Gapless traversing is achieved by either ray-casting or line-drawing. In the first case the parametric equation of the ray is intersected with the analytical equations of the voxel boundaries. The results are very accurate, but also very slow. In the second case, traversing is executed in the discrete voxel space, and line coherence as well as integer arithmetic are exploited to speed-up computations. A Bresenham-like line algorithm is edge-connected, and thus, will skip several 'touched' voxels ([SaGe91b], see fig. 1). Therefore, a face-connected line algorithm is needed, see [Slat92] and [EnSo94] for a detailed discussion. The algorithms presented in [ClWy88] and [YaCK92] both fulfill this requirement. [YaCK92] is faster, but provides only integer voxel coordinates; therefore [ClWy88] is used. In addition, the coordinates of the ray intersection with the voxel boundaries at entry and exit position as real numbers are needed for the computation of the maximum as explained below in more detail (see fig. 2). The [ClWy88] algorithm fulfils this requirement without any analytical computations by exploiting the coherency.

However, two extensions that are important for MIP applications are necessary: first, in the original version, the termination criterion is based on a numerical comparison of the accumulated and the end-point coordinates. Such a termination can easily fail because of error accumulations, especially if it is implemented in fix point arithmetic and for long ray-paths. Additional tolerance criteria significantly slow down the all-round performance. We modified the termination criterion by calculating the total number of steps along each ray in advance just by comparing the total ray length to the voxel edge divided by the cosine of the ray angle. With this new termination criterion we were able to implement the algorithm in fix point arithmetic while avoiding error accumulations.

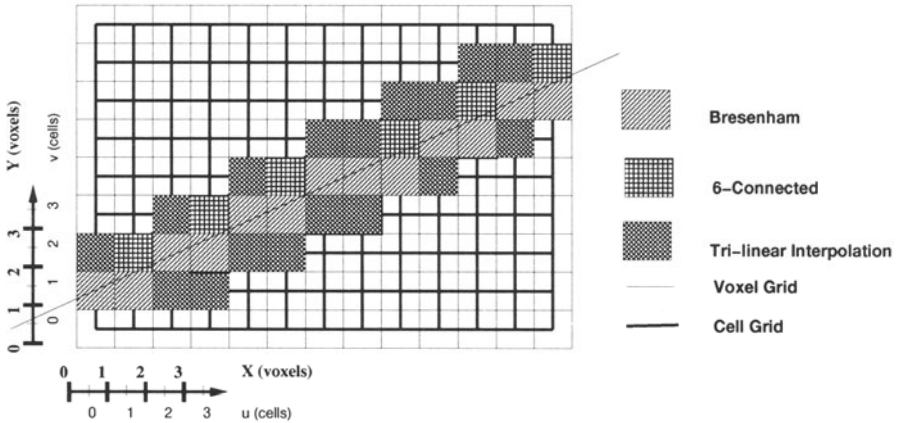


Figure 1: Voxels selected by different line-traversing methods

Even using the [ClWy88] algorithm, a comparison of the voxels selected by the face-connected line in fig. 1 and the voxels contributing to the ray shows that even this method will not select all required voxels. The reason is that the trilinear interpolation addresses all 8 voxels around the position selected on the ray. We

refer to the cube defined by the midpoints of 8 neighbouring voxels an 'interpolation cell', or simply 'cell'. Within such a cell the value varies in space, but the expression of the interpolation function remains unchanged. It can easily be seen in fig. 1 that the cell grid is simply translated by a half voxel along the u, v, w directions of the voxel grid. Its resolution equals the voxel resolution minus one.

In order to address all contributing voxels in an effective way, we decided to traverse the ray on the cell grid instead on the voxel grid. Thus, with each step the line penetrates boundaries of interpolation domains instead of voxel boundaries. If a cell is selected, all of its 8 contributing voxels are selected. The complete ray-path is divided into a chain of individual segments, each of them is defined by the intersection with the ray with the cell boundaries. The density within each segment is given by a polynomial of third degree with coefficients differing between cells as expressed in eq. 7. Due to the usage of a face-connected line, each new step has 4 voxels in common with the previous one and addresses only 4 new additional voxels. Buffering the voxels helps decreasing access time.

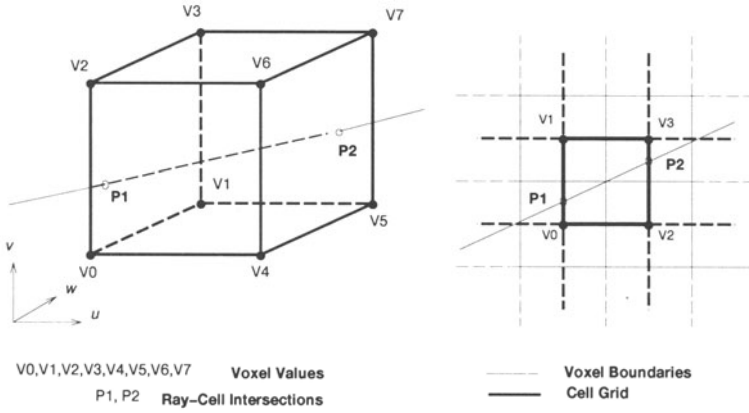


Figure 2: Intersection of a ray with the cell-grid

4 Cell Pre-Selection

Once the cells and their corresponding 8 voxels contributing to a line segment are identified, the maximum within the cell – a candidate for the global ray-maximum – has to be calculated. A simple observation is, that the maximum value within a cell after interpolation will always be smaller than the maximum value of the 8 voxels (or equal to that, if all voxels are identical). Now, since we are looking for the maximum along a ray, the maximum of the 8 neighbours of the newly selected cell is compared to the intermediate maximum of the ray. If none of the 8 voxels is greater than the current ray value, the cell (i.e., 8 voxels) can be skipped without further processing. Finding the maximum of a cell requires 8 voxel accesses and 7 integer comparisons. As an alternative, the maximum of each cell can be found during pre-processing and can be stored together with the data. In this case, the cell-grid is traversed using a 6-connected line. For each new cell the corresponding maximum from the pre-processed field is selected, and is compared with the current ray maximum. Thus, a single nearest neighbour integer comparison is performed instead of 7 at a single step for the cost of additional memory space. Note, that for storing the cell maximum only a pointer to the corresponding voxel is needed, i.e. a number between 0 and 7. Such a pointer requires only additional 3 bits per

cell. Since MR and CT datasets usually have 12 significant bits and a voxel is typically allocated as a short integer (16 bits), these 3 pointer bits can be stored in the remaining 4 bits, and therefore, require no additional space at all.

Tests with two dozen different datasets (CT, MR, 3D-ultrasound, nuclear medicine etc.) using the comparison criterion mentioned above showed that roughly 85%-95% of the cells can be skipped without any further processing. Due to the specific characteristics of MRA datasets, the savings in this particular case are even higher and are between 90% and 99%, so that only 1% – 10% of the total cell evaluations have to be performed.

5 Maximum Value Calculation

Once an interesting cell has been selected, i.e., a cell with one of the 8 voxels higher than the current ray maximum, the maximum within this cell has to be found. Neither uniform or adaptive oversampling nor binary search are appropriate approaches since they both require several trilinear interpolations, that is by far the most expensive operation of the complete MIP process. To overcome this difficulty, we developed two different methods to find the maximum without any oversampling. For better discrimination between the voxels and cells, voxel coordinates will be referred to as (x,y,z) and the cell coordinates as (u,v,w) .

The first method is based on the idea that the maximum voxel of the cell will most probably have the most significant influence on the value of the interpolation, i.e., the desired maximum value will most probably be on a ray location close to the value of the maximum of the 8 cell voxels. The point on the ray closest to this maximum voxel can be found at the intersection of the ray and the perpendicular going through the corresponding voxel midpoint, see fig. 3. C is the voxel midpoint. AB the ray segment and CK the perpendicular. The (x,y,z) coordinates of the point K can be calculated by exploiting coherence along the ray as stated below.

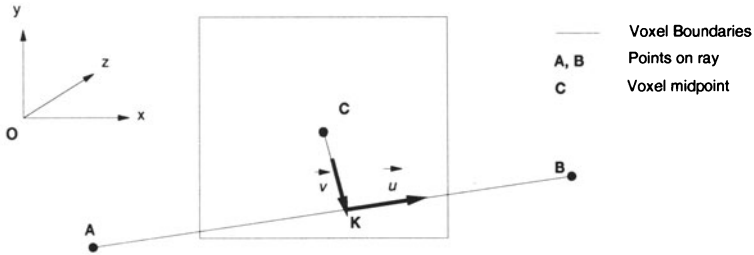


Figure 3: Estimating the perpendicular from the voxel midpoint C on the ray AB

We investigate the points $A(a_1, a_2, a_3)$, $B(b_1, b_2, b_3)$, $C(c_1, c_2, c_3)$, and $K(x_k, y_k, z_k)$. O is the origin of the coordinate system. For $\vec{u}$ perpendicular on $\vec{v}$, their scalar product has to be $\vec{u} \cdot \vec{v} = 0$. From fig. 3, $\vec{u} = \vec{OB} - \vec{OA}$ and $\vec{v} = \vec{OK} - \vec{OC}$. For $\vec{OK}$ we have:

$$\vec{OK} = \vec{OA} + \lambda \cdot (\vec{OB} - \vec{OA}) \quad (1)$$

The scalar product $\vec{u} \cdot \vec{v} = 0$ can be written as

$$(\vec{OB} - \vec{OA}) \cdot (\vec{OA} + \lambda \cdot (\vec{OB} - \vec{OA}) - \vec{OC}) = 0 \quad (2)$$

and after substitution of the coordinates:

$$\begin{aligned} & (b_1 - a_1) \cdot (a_1 + \lambda \cdot (b_1 - a_1) - c_1) + \\ & (b_2 - a_2) \cdot (a_2 + \lambda \cdot (b_2 - a_2) - c_2) + \\ & (b_3 - a_3) \cdot (a_3 + \lambda \cdot (b_3 - a_3) - c_3) = 0 \end{aligned} \quad (3)$$

As a result from solving the above parametric equation, λ is

$$\lambda = \frac{(c_1 - a_1) \cdot (b_1 - a_1) + (c_2 - a_2) \cdot (b_2 - a_2) + (c_3 - a_3) \cdot (b_3 - a_3)}{(b_1 - a_1)^2 + (b_2 - a_2)^2 + (b_3 - a_3)^2} \quad (4)$$

and the coordinates of K are:

$$\begin{aligned} x_k &= a_1 + \lambda \cdot (b_1 - a_1) \\ y_k &= a_2 + \lambda \cdot (b_2 - a_2) \\ z_k &= a_3 + \lambda \cdot (b_3 - a_3) \end{aligned} \quad (5)$$

For finding the coordinates of K only the current value of λ has to be calculated. The values of $(b_1 - a_1)/((b_1 - a_1)^2 + (b_2 - a_2)^2 + (b_3 - a_3)^2)$, $(b_2 - a_2)/((b_1 - a_1)^2 + (b_2 - a_2)^2 + (b_3 - a_3)^2)$, and $(b_3 - a_3)/((b_1 - a_1)^2 + (b_2 - a_2)^2 + (b_3 - a_3)^2)$ have to be calculated only once per ray, exploiting coherence. In parallel projections these values have to be calculated once per image. For each selected cell the maximum among its 8 voxels is identified and the position of point K is calculated from eq. 4 and eq. 5. The density value $\rho(K)$ is found by trilinear interpolation. As a result, only a single interpolation is required per cell. Although an approximation, the images calculated by this method provide results sufficient for most applications (see section 7).

The second method calculates position and value of the maximum analytically, thus, involves no approximation at all. We first calculate the interpolation function giving the density along the ray segment within each cell. We used the package *Mathematica* to compute the corresponding expressions. The 8 voxels on the corners of the cell have the values $V_0, \dots, V_7$. By writing the ray-equation in parametric form and substituting in the equation of trilinear interpolation, the equation for the density along the ray becomes a polynomial of third degree $\rho(t^3, t^2, t, V_0, V_1, \dots, V_7)$. The value of t varies from 0.0 to 1.0 along the ray segment P_1P_2 ¹. The coordinates (u_1, v_1, w_1) for the entry and (u_2, v_2, w_2) for the exit point in the cell are normalized ($[0.0, 1.0]$) relative to the lower left corner of the cell, i.e., $u = x - \lfloor x \rfloor + 0.5$, see also fig. 1. We first define the following values:

$$\begin{aligned} s_1 &= -V_0 + V_1, & s_2 &= -V_0 + V_2, & s_3 &= V_0 - V_1 - V_2 + V_3 \\ s_4 &= -V_0 + V_4, & s_5 &= V_0 - V_1 - V_4 + V_5 \\ s_6 &= V_0 - V_2 - V_4 + V_6 \\ s_7 &= -V_0 + V_1 + V_2 - V_3 + V_4 - V_5 - V_6 + V_7 \\ \Delta u &= u_2 - u_1, & \Delta v &= v_2 - v_1, & \Delta w &= w_2 - w_1 \\ \Delta uv &= \Delta u \cdot \Delta v, & \Delta vw &= \Delta v \cdot \Delta w, & \Delta uw &= \Delta u \cdot \Delta w \\ A &= \Delta u \cdot \Delta v \cdot \Delta w \cdot s_7 \\ B &= \Delta uv \cdot (s_6 + s_7 \cdot w_1) + \Delta uw \cdot (s_5 + s_7 \cdot v_1) + \\ &\quad \Delta vw \cdot (s_3 + s_7 \cdot u_1) \\ C &= s_7 \cdot (u_1 \cdot v_1 \cdot \Delta w + u_1 \cdot \Delta v \cdot w_1 + \Delta u \cdot v_1 \cdot w_1) \end{aligned}$$

¹Note that the value of t varies always between 0.0 and 1.0, independent of the length of the segment P_1P_2

$$\begin{aligned}
& +s_6 \cdot (u_1 \cdot \Delta v + \Delta u \cdot v_1) + s_5 \cdot (u_1 \cdot \Delta w + \Delta u \cdot w_1) \\
& +s_3 \cdot (v_1 \cdot \Delta w + \Delta v \cdot w_1) + s_4 \cdot \Delta u \\
& +s_2 \cdot \Delta v + s_1 \cdot \Delta w \\
D = & u_1 \cdot v_1 \cdot w_1 \cdot s_7 + u_1 \cdot v_1 \cdot s_6 + u_1 \cdot w_1 \cdot s_5 + \\
& v_1 \cdot w_1 \cdot s_3 + u_1 \cdot s_4 + v_1 \cdot s_2 + w_1 \cdot s_1 + V_0
\end{aligned}$$

Using the above quantities, the equation for the trilinear interpolation and its first and second derivatives yields to:

$$\rho(t^3, t^2, t, V_0, V_1, \dots, V_7) = A \cdot t^3 + B \cdot t^2 + C \cdot t + D \quad (6)$$

$$\rho'(t^2, t, V_0, V_1, \dots, V_7) = 3 \cdot A \cdot t^2 + 2 \cdot B \cdot t + C \quad (7)$$

$$\rho''(t, V_0, V_1, \dots, V_7) = 6 \cdot A \cdot t + 2 \cdot B \quad (8)$$

Within each cell the density function has a local maximum either at the location t where the first derivative $\rho'(t)$ is equal to zero and the second derivative $\rho''(t)$ is negative, or at the cell boundaries $t = 0$ or $t = 1$. In order to optimize the process and avoid unnecessary calculations, the calculations are organized in a hierarchy of growing complexity as explained below. First we calculate the determinant of eq. 7

$$4 \cdot B^2 - 4 \cdot 3 \cdot A \cdot C = 4 \cdot (B^2 - 3 \cdot A \cdot C) = 4 \cdot \Delta \quad (9)$$

If the determinant is less than zero ($\Delta < 0$), a real valued solution does not exist for eq. 7, thus, the maximum has to be one of the two boundary values. After substituting the values $t = 0$ and $t = 1$ in eq. 7 we find $\rho(0) = D$ and $\rho(1) = A + B + C + D$. Therefore, the cell maximum is $\rho(0)$, if $A + B + C < 0$ and $\rho(1)$ otherwise.

If $\Delta \geq 0$, real valued solutions of eq. 7 are possible. Solving the quadratic equation we get two possible values:

$$t_{1,2} = \frac{-2 \cdot B \pm \sqrt{4 \cdot \Delta}}{2 \cdot 3 \cdot A} = \frac{-B \pm \sqrt{\Delta}}{3 \cdot A} \quad (10)$$

Substituting $t_{1,2}$ in eq. 8 and solving for $\rho''(t) < 0$ we get:

$$\begin{aligned}
\rho''(t_1) < 0 &\Rightarrow 6 \cdot A \cdot \frac{-B + \sqrt{\Delta}}{3 \cdot A} + 2 \cdot B < 0 \Rightarrow \sqrt{\Delta} < 0 \\
\rho''(t_2) < 0 &\Rightarrow 6 \cdot A \cdot \frac{-B - \sqrt{\Delta}}{3 \cdot A} + 2 \cdot B < 0 \Rightarrow -\sqrt{\Delta} < 0
\end{aligned} \quad (11)$$

Obviously, the first requirement is always false and the second always true. Thus, a local maximum exists only for t_2 . We further discriminate three different cases as illustrated in fig. 4. In the first case t_2 is outside the interval $[0,1]$, i.e., the maximum is not located within the section P_1P_2 . In this case cell maximum is calculated on one of the boundaries as in the previous case.

In the second case a true local maximum exists, in the third case a boundary value is larger than the local maximum. For discriminating between cases two and three, we further calculate t_1 . If $t_2 \in [0, 1]$ and $t_1 \notin [0, 1]$, case two applies and the desired maximum is $\rho(t_2)$. In the opposite case where $t_2 \in [0, 1]$ and $t_1 \in [0, 1]$, the local maximum must be compared to the maximum of the boundary values. In this case, we first calculate the maximum boundary value as stated above and then compare it to $\rho(t_2)$.

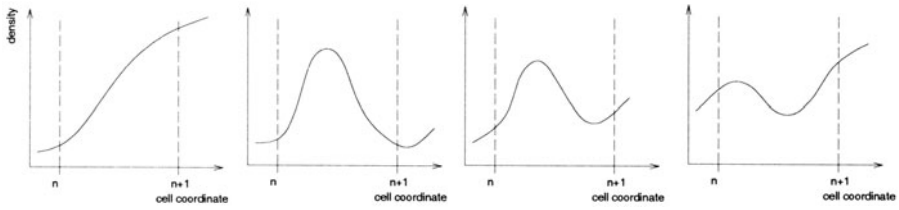


Figure 4: From left to right: no local maximum within the cell; true local maximum; boundary value may be smaller or larger than the local maximum

6 Improving Cache Coherency

After the number of interpolations for a single ray is reduced to a minimum, most of the remaining computation time during traversing is spent for the calculation of the coordinate of the next step and the memory access of the voxel values. Since for preview and approximat methods the traversing is a simple (and fast) Bresenham-like Fixpoint-DDA, the memory access time becomes an extremely important issue.

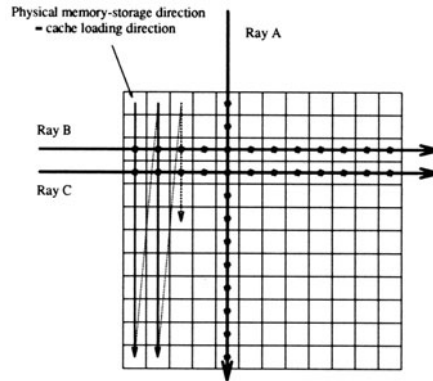


Figure 5: Cache problems - Ray A: Best case Ray B: worst case

The main observation from the evaluation of runtime results is that the rendering time depends strongly on the traversing direction even if the total number of voxel-accesses remains constant. Fig. 5 shows two typical situations (in 2D) when a ray traverses the volume. Ray A traverses the volume in direction of the physical memory allocation, i.e. in the direction of increasing addresses. For each access to the memory the operating system initially tries to load the data from the cache² memory, and then, if the required data is missed, from the main memory. In the case of a cache-fault the system will load additional bytes into the cache assuming that the next memory access will most likely be close to the first request. For Ray A this works fine, but it fails for Ray B, because the next traversing step does not access a pre-loaded cache-line. In this case, the system will always reload the cache memory resulting in heavy bus-traffic resulting in increasing rendering times.

²For shortness, the term 'cache' refers to the first & second level cache

It is important to note that the systems cache-loading mechanism typically works like a LRU-list: new cache-lines will remove the least recently used cache-line in the cache memory. Therefore, it depends on the viewing direction, volume resolution and cache-size whether or not a neighbour ray (Ray C) will traverse pre-loaded cache-memory. However, the typical situation for large volumes ($> 128^3$) is that the rays loaded into the cache will be replaced by new ones before a neighbour ray accesses these lines again. This increases the number of cache-faults and, therefore, the bus-traffic, that is a serious performance degradation factor on shared memory multi-processor platforms.

One possibility to work around this problem is to change the traversing method from the image to the object space (splatting). This can be done in (linear) voxel-space and guarantees a maximum cache coherence. However, this advantage of the splatting method is usually wasted since the evaluation of voxel-influences for the covered pixels - even for a 'preview like' MIP - is very computational demanding.

A better solution of this problem surprisingly is as simple as effective: the linear volume-allocation is changed to a sub-cube representation (see fig. 6) in order to increase the locality of the address-voxelspace. The volume is still allocated in one piece; this method requires only changes in the voxel memory-address calculation.

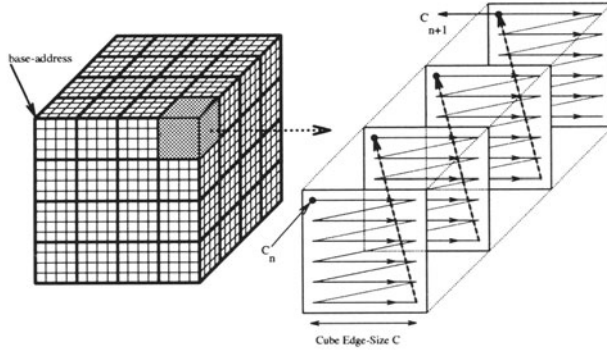


Figure 6: Sub-cube Memory Organization

Let C be the edge-size of one sub-cube, (u,v,w) the voxel-coordinate and $(U_{res}, V_{res}, W_{res})$ the total resolution of the volume. We first compute the resulting address-offset C_n relative to the starting address (*base-address*):

$$C_n(u, v, w) = ((w \text{ DIV } C) * (U_{res} \text{ DIV } C) * (V_{res} \text{ DIV } C) + (v \text{ DIV } C) * (U_{res} \text{ DIV } C) + (u \text{ DIV } C)) * C^3 \quad (12)$$

With C_n and the allocated base address the physical memory address of a voxel becomes

$$Address(u, v, w) = base-address + C_n(u, v, w) + (w \text{ MOD } C) * C^2 + (v \text{ MOD } C) * C + (u \text{ MOD } C) \quad (13)$$

If all constant values are fractions of 2, these calculations can be done with (fast) SHIFT and AND operations, but for arbitrary volume-sizes and/or cube-sizes a complete calculation, including modulo, multiplications and divisions, is necessary - obviously to time-consuming for our purpose. In order to speed up the address

calculation a 'Look Up Table' (LUT) mechanism is implemented. With Eq. 12 and Eq. 13 the address-calculation can be rewritten as

$$Address(u, v, w) = G(u) + H(v) + I(w)$$

with

$$\begin{aligned} G(u) &= (u \text{ DIV } C) * C^3 + (u \text{ MOD } C) \\ H(v) &= (v \text{ DIV } C) * (U_{res} \text{ DIV } C) * C^3 + (v \text{ MOD } C) * C \\ I(w) &= (w \text{ DIV } C) * (U_{res} \text{ DIV } C) * (V_{res} \text{ DIV } C) * C^3 + \\ &\quad (w \text{ MOD } C) * C^2 + base-address \\ base-address &= MALLOC(sizeof(char) * U_{res} * V_{res} * W_{res}) \end{aligned}$$

These values can be precalculated and stored in three look-up tables, thus, the memory access appears in 'C'-code like

```
value = *((unsigned char *) (G[u] + H[v] + I[w]));
```

The size of the LUT's depends on the resolution of the allocated volume. For example, a resolution of 256^3 results in $3 * 256 * 4 = 3072$ bytes of additional memory space. Since this is very small - compared to typical cache sizes (e.g. 256k byte) - will not affect the cache load-balance.

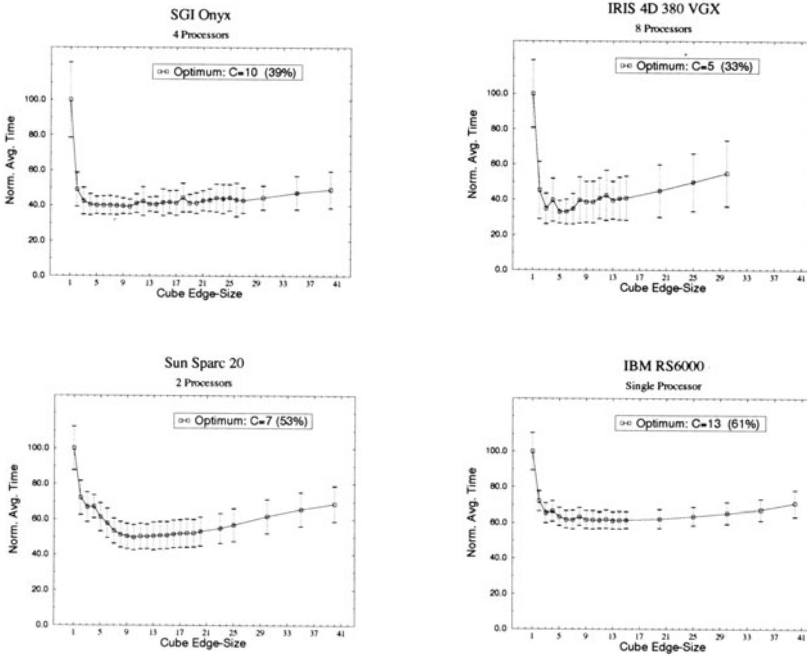


Figure 7: Average rendering times as a function of the cube edge-size

In fig. 7 we present the results of different machines of this cache-optimized memory organization. All measured rendering times are averaged over 100 different viewing

directions using preview quality (Bresenham Fixpoint DDA, one nearest neighbour access step). For clarity, all times are normalized with respect to the average rendering time that is achieved if the linear memory organization is employed (cube-size = 1). For all tested platforms there is a different optimal cube-size that provides the best cache coherency. Additionally, the direction-dependency of rendering times is minimized as well for this particular cache size. The achieved speed-up varies between 2 and 3.

7 Results

The following table presents some runtimes for four different datasets and varying methods and image qualities. The image resolution is 512^2 and the volume covers approximately 70% of the pixels. The datasets represent different applications: the first dataset is a nuclear medicine examination, the second and third are MRA's, and the fourth is a 3D ultrasonic dataset.

	64^3	$256^2 \cdot 63$	$256^2 \cdot 165$	256^3
Nearest Neighbour	4.78	9.3	11.2	15.3
DDA & max. approximation	16.2	29	32.6	39.2
Cell traversing & approximation	38.3	95.7	79.5	94.6
Cell traversing & analytic	61.1	154.9	105.4	124.7

Table 1: Runtimes in CPU seconds on a SGI R4000 for four different datasets and varying image qualities

The second table presents some statistics about the various methods measured with the third dataset. The 1st row shows the fastest method (26-connected line, one access for a single step, no interpolation). The 2nd row is a 6-connected line with the maximum approximation of eq. 5. The 3rd and 4th rows are calculated with cell-traversing where the maximum is either approximated or analytically computed. Clearly, it shows that only 3%-10% of the addressed locations are evaluated; the rest is skipped without further processing.

	Nearest Neighbour	DDA traversing & approx.	Cell traversing & approx.	Cell traversing & analytic
# steps	3,580,000	6,800,000	6,800,000	6,800,000
Steps/ray	127.4	242	242	242
Voxel accesses	3,580,000	8,950,000	54,400,000	54,400,000
MIP evaluations	78,730 (2.2%)	195,600 (2.9%)	787,500 (11.6%)	732,670 (10.7%)
MIP/ray	2.8	7.0	28.0	26.1

Table 2: Statistics measured on the third dataset for varying image quality

The following images illustrate the results of this paper. In fig. 8, we rendered a 'phantom volume' containing straight lines of the thickness of one voxel. Fig. 9 shows an MRA example (the 2nd dataset). The approximation of the maximum value using the perpendicular gives remarkably good results. In fact, differences between the accurate and the approximation methods can be shown clearly only on the specially generated phantom dataset, indicating the efficiency of the approximation.

8 Conclusion

The impetus for this research was the observation that usually over 90% of the computation during MIP calculations is wasted. We speeded up this process significantly by applying three different optimizations.

First, we introduced a cell-traversing scheme for gapless and effective selection of all voxels contributing to a ray. We further extended the algorithm published in [ClWy88] to operate with fix point arithmetic while avoiding the accumulation of numerical errors.

Second, by comparing the newly accessed voxel values with the current maximum of the ray, we avoided further processing for more than 90% of the traversed cells.

Third, we developed two different methods for calculating the maximum value within a selected cell. The first employs a simple, but very effective, approximation for the estimation of the position and value of the maximum with an accuracy sufficient for most applications, the second utilizes an accurate analytical solution. Both methods avoid oversampling and require only a single interpolation for a single selected cell.

Literature

- ClWy88 Cleary, J., Wyvill, G.: *Analysis of an Algorithm for Fast Ray-Tracing Using Uniform Space Subdivision*, "The Visual Computer", Vol. 4, pp. 65-83, 1988
- EnSo94 Endl, R., Sommer, M.: *Classification of Ray-Generators in Uniform Subdivisions and Octrees for Ray Tracing*, Computer Graphics Forum, Vol. 13, No. 1, pp. 3-19, March 1994
- Rouf91 Roufchaie, V. (Edt.): *Power Series Performance Guide*, Technical Report Power Series, Silicon Graphics, 1991
- SaGe91b Sakas, G., Gerth, M.: *Sampling and Anti-Aliasing Discrete 3D Volume Density Textures*, EUROGRAPHICS'91 Award Paper, Computer and Graphics, Vol. 16, No. 1, pp. 121-134, Pergamon Press, 1992
- Saka93d Sakas, G.: *Interactive Volume Rendering of Large Fields*, "The Visual Computer", Vol. 9, No. 8, pp. 425-438, August 1993
- Slat92 Slater, M.: *Tracing a Ray Through Uniformly Subdivided n-Dimensional Space*, "The Visual Computer", Vol. 9, pp. 39-46, 1992
- Vand91 Vandermeulen, D.: *Methods for Registration, Interpolation and Interpretation of 3D Medical Image Data for use in 3-D Display, 3-D Modeling and Therapy Planning*, Ph.D. Thesis, Katholieke Universiteit Leuven, Faculteit Toegepaste Wetenschappen, ESAT/MI2, U.D.C. 681.3 I4:616-089, April 1991
- YaCK92 Yagel, R., Cohen, D., Kaufman, A.: *Discrete Ray Tracing*, IEEE Computer Graphics & Applications, September 1992

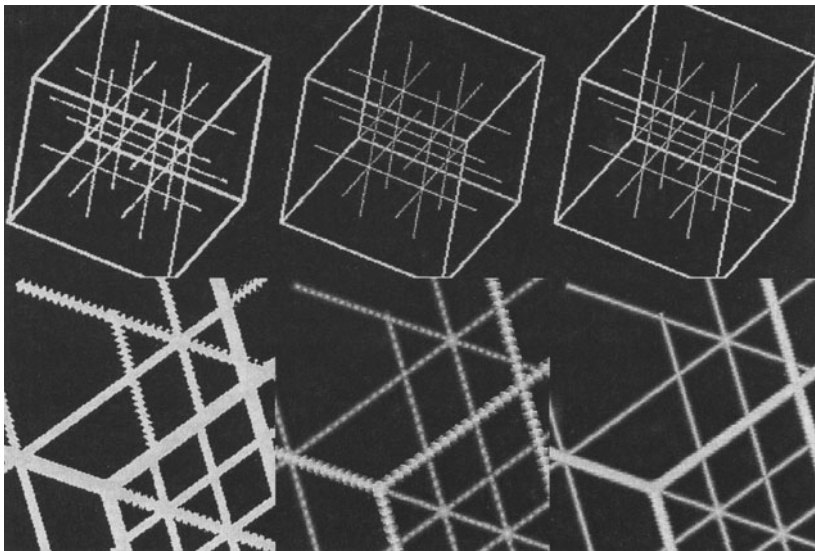


Figure 8: Rendering a phantom volume with preview, approximation, and accurate methods. Magnifications are shown in the lower row.

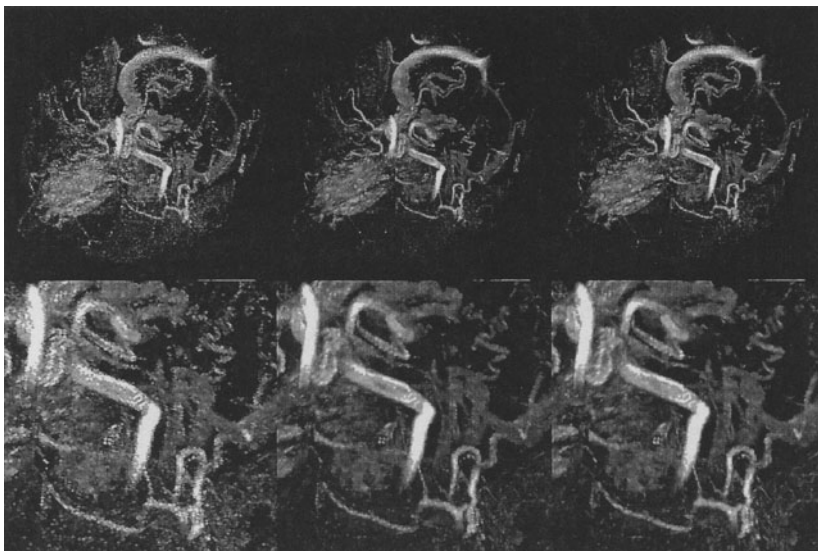


Figure 9: Rendering an MRA set ($256^2 \cdot 63$) with preview, approximation, and accurate method. Magnifications are shown in the lower row. Courtesy of Dr. Meindl, DKD-Wiesbaden